SETU 2026 / SWFW_5S_12

Distributed Linux Fleet Management

Project implementation report

A web-based controller for Linux node onboarding, SSH availability checks, controlled diagnostics and Prometheus–Grafana monitoring.

1 Student Details

Field	Details
Student name	Adarsh Singh
Institution	Government College of Engineering, Aurangabad
Program	B.Tech in Computer Science and Engineering
Year and semester	Third year · Fifth semester
Domain	Software / Firmware
Email	bt24f05f057@geca.ac.in

2 Project Details

Field	Details
Problem statement	SWFW_5S_12
Project title	Distributed Linux Fleet Management
Report date	12 November 2026
Technology	Python 3.12 · FastAPI · Paramiko · SQLite WAL Prometheus · Grafana · Browser ES modules

The implementation separates host identity, connection observations and resource telemetry. It records remote diagnostic execution before dispatch and preserves the operation history across controller restarts.

REPORT NAVIGATION

Contents

3.1 Scope and delivered functionality 4
3.2 System context and ownership 5
3.3 Deployment and process boundaries 6
3.4 Internal structure and request handling 7
3.5 Node onboarding and identity enrollment 8
3.6 Trust consistency and configuration revisions 9
3.7 Availability scheduling and failure isolation 10
3.8 Availability and freshness interpretation 11
3.9 Controlled diagnostic catalog 12
3.10 Dispatch and the durable audit barrier 13
3.11 Completion persistence and restart recovery 14
3.12 Metrics collection and visualization 15
3.13 Metric validity and source timestamps 16
3.14 Relational model and storage ownership 17
3.15 Authentication and browser sessions 18
3.16 Security controls and engineering limits 19
3.17 Backup and recovery procedures 20
3.18 Operator workflow and interface behavior 21
4.1 Inventory and node setup 22
4.2 Diagnostic workbench 23
4.3 Native resource trends 24
4.4 Missing measurements 25
5.1 Test strategy and recorded results 26
5.2 Monitoring and admission measurements 27
6 Design decisions and tradeoffs 28
Appendix A Deployment and reproduction 29
Appendix B Implementation traceability 30
Appendix C References and terminology 31

REPORT NAVIGATION

Figures and reading paths

01 System context and independent control and telemetry paths 5
02 Deployment boundaries and process ownership 6
03 Controller module layers and dependency direction 7
04 Onboarding sequence with separate server trust and client authorization 8
05 Scheduler timeline separating collection from read time freshness 10
06 Ordered availability decision flow 11
07 Operation sequence showing the durable audit barrier before dispatch 13
08 Operation outcome model and interrupted execution reconciliation 14
09 Telemetry data flow and separate visualization clients 15
10 Source time and query time with explicit missing data 16
11 Relational data model with exact ownership and cardinalities 17
12 Authentication branches and browser mutation gates 18
13 Paired backup and restore procedures with independent asset boundaries 20
14 Measured admission and cleanup results 27
15 Inventory browser view 22
16 Diagnostic workbench 23
17 Native Fleet Health trends 24
18 Missing-data presentation 25

A focused evaluator reading path

Begin with the requirements and system context. Follow host enrollment, then the operation dispatch sequence and completion recovery. Read telemetry semantics and the data model before reviewing the screenshots and measured results.

Diagram conventions: blue denotes controller logic or records; teal denotes telemetry or completed steps; amber denotes a gate or limit. Labels carry the meaning independently of color. Sequence arrows show event order; data-model arrows show relationships; timelines are explicitly illustrative unless marked measured.

3.1 Scope and delivered functionality

Fleet Manager centralizes the repeated work of registering Linux machines, establishing SSH trust, observing connectivity, running bounded diagnostics and viewing resource measurements. The administrator uses a browser dashboard or authenticated API; the managed machines provide sshd and Node Exporter. No custom management agent is installed on the hosts.

Requirement	Implemented mechanism	Operator result
Onboard and manage nodes	Validated inventory, explicit host-key enrollment and authenticated SSH checks	A persistent node record with separate setup and observation states
Observe availability	Scheduled, bounded checks with timestamps and classified failures	Current reachability, stale observations and actionable error categories
Execute controlled commands	Four-action catalog, typed arguments, admission limits and durable history	Bounded output linked to a specific node and operation ID
Collect and visualize metrics	Node Exporter, Prometheus storage, Grafana provisioning and native health views	Resource measurements with target and source-time context

System scope

The reference deployment uses one controller worker, one administrative identity and a three-node Linux fleet. The controller is a modular application with isolated child processes for transport work. Prometheus and Grafana run as separate services. A three-node demonstration describes the deployment used; it is not a capacity ceiling.

Design boundaries

The command interface provides diagnostics rather than an unrestricted shell. The baseline does not include remote service repair, multi-administrator RBAC, a highly available controller or distributed job ownership. Resource telemetry and SSH availability remain separate signals, allowing an exporter failure to be distinguished from an authentication or network failure.

Report basis: supplied reports (5) and (6); project charter and module paths reproduced in Appendix B. Implementation and testing status is confirmed by the project author. Quantitative results retain their recorded dates and test environments.

3.2 System context and ownership

The controller owns inventory, SSH trust enrollment, connectivity observations and diagnostic history. Prometheus owns resource time series. Grafana reads those time series from Prometheus, while the controller exposes a fixed-query health projection to its own browser interface.

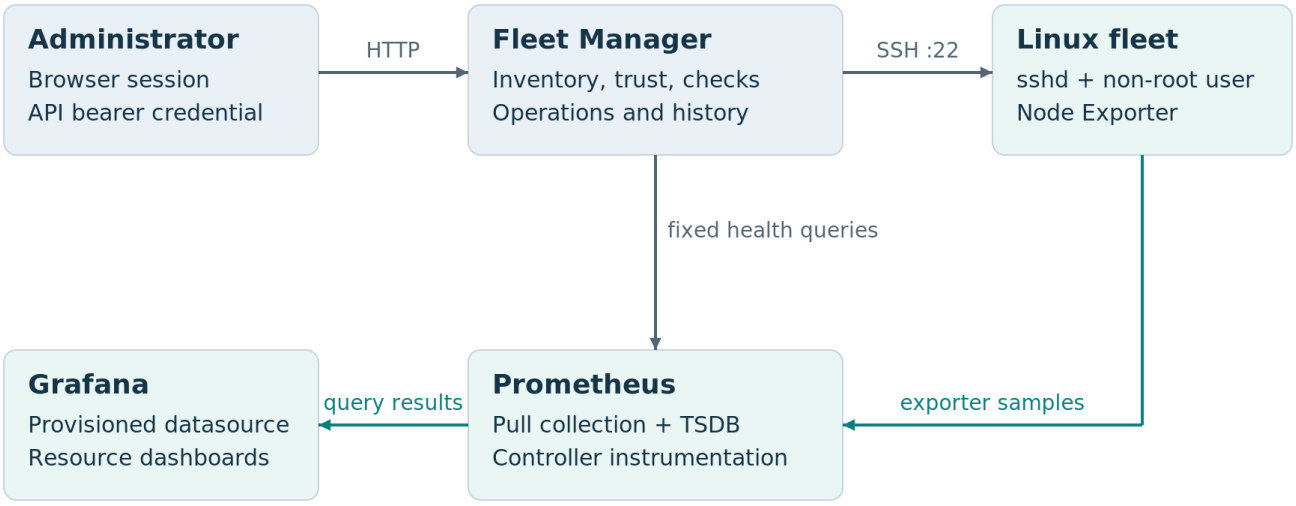


Figure 1. System context and independent control and telemetry paths.

Why the paths are independent

A successful SSH connection establishes that the endpoint responded using the expected identity and credentials. It does not establish that Node Exporter is reachable or that memory usage is acceptable. Conversely, a healthy exporter can continue to provide metrics while SSH authentication fails. The interface preserves those distinctions instead of collapsing them into one machine-health flag.

Control and data ownership

Inventory changes affect which nodes the controller can address. They do not automatically authorize the controller key on a host or prove that an exporter target is uniquely configured. Each boundary has its own identity, configuration and failure handling.

Implementation anchors: main.py; inventory, connectivity, operations and metrics modules; monitoring/ configuration. Diagram structure follows the separation of static and runtime views in reference R1.

3.3 Deployment and process boundaries

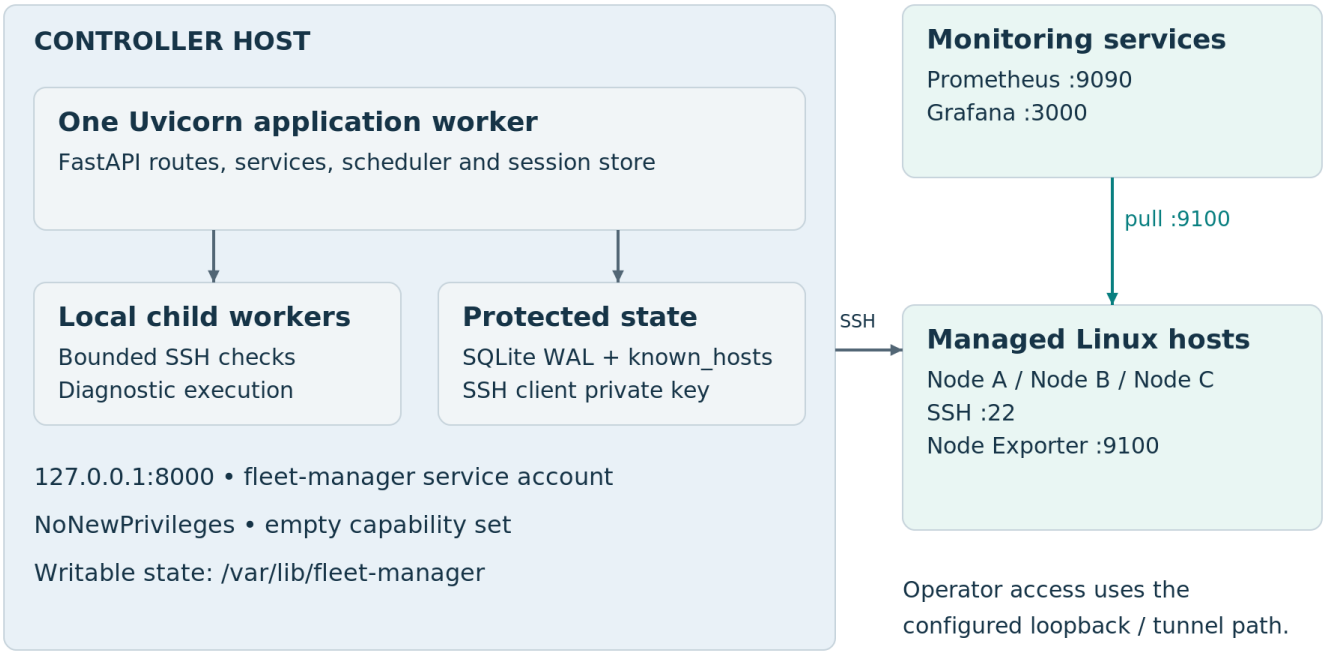


Figure 2. Deployment boundaries and process ownership.

The systemd unit starts Uvicorn at 127.0.0.1:8000 with exactly one application worker. The service runs as fleet-manager, with NoNewPrivileges, an empty capability set, read-only configuration and writable state under /var/lib/fleet-manager. Managed machines expose SSH and Node Exporter through their configured network paths.

Why one worker is required

Sessions, admission tracking and the scheduler are process-local. Starting additional application workers would split session state, duplicate scheduling and break operation ownership. Startup reconciliation could also mark another worker's active operation UNCERTAIN. Scaling requires shared ownership and coordination before increasing the worker count.

Grafana uses port 3000 and Prometheus uses port 9090. The dashboard's loopback Grafana link requires a corresponding operator tunnel or deployment configuration. Loopback on the operator's browser identifies the operator's machine, not the remote controller.

Implementation anchors: deploy/fleet-manager.service; main.py lifespan; ui/templates/dashboard.html. Deployment notation: R1.

3.4 Internal structure and request handling

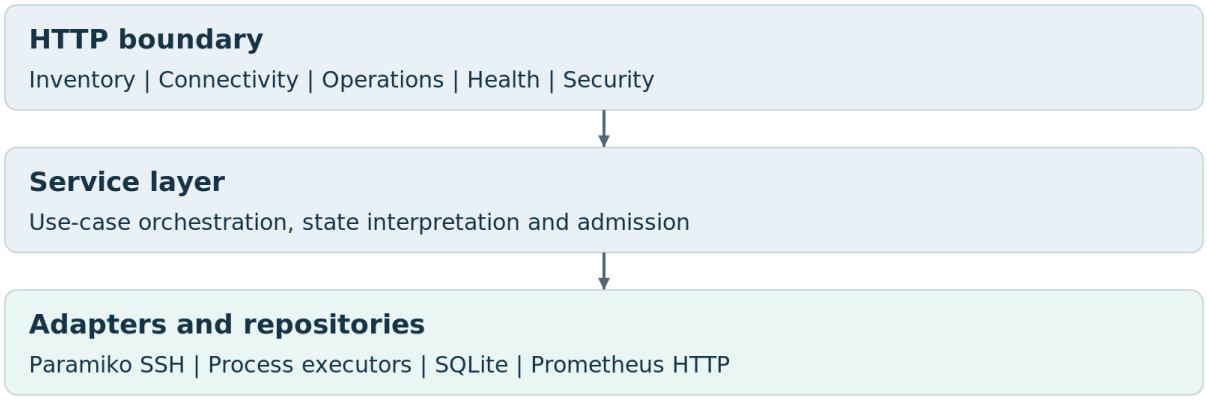


Figure 3. Controller module layers and dependency direction.

HTTP boundary

Routers authenticate protected requests, validate structured input and translate service outcomes into HTTP responses. The operation API accepts an action and arguments rather than a user-supplied shell program. A successful HTTP response can contain a failed diagnostic result; transport status and domain outcome have separate meanings.

Services and adapters

Services coordinate inventory lookup, configuration revisions, admission, audit persistence and result classification. Repositories own parameterized SQL and short transactions. Transport adapters handle SSH or Prometheus requests. Child workers isolate blocking transport execution within a locally enforceable lifetime.

Application lifecycle

Startup initializes repositories, checks schema state, reconciles interrupted RUNNING operations and starts the scheduler when enabled. Shutdown stops scheduling and performs bounded cleanup of active work. The child-process boundary makes local cleanup possible without turning the controller into a distributed queue.

Implementation anchors: main.py; connectivity/scheduler.py; operations/executor.py; security/session.py; repository modules.

3.5 Node onboarding and identity enrollment

Onboarding establishes three separate facts: the endpoint is registered, the server identity is trusted, and the controller is authorized to authenticate. Each has its own step and result.

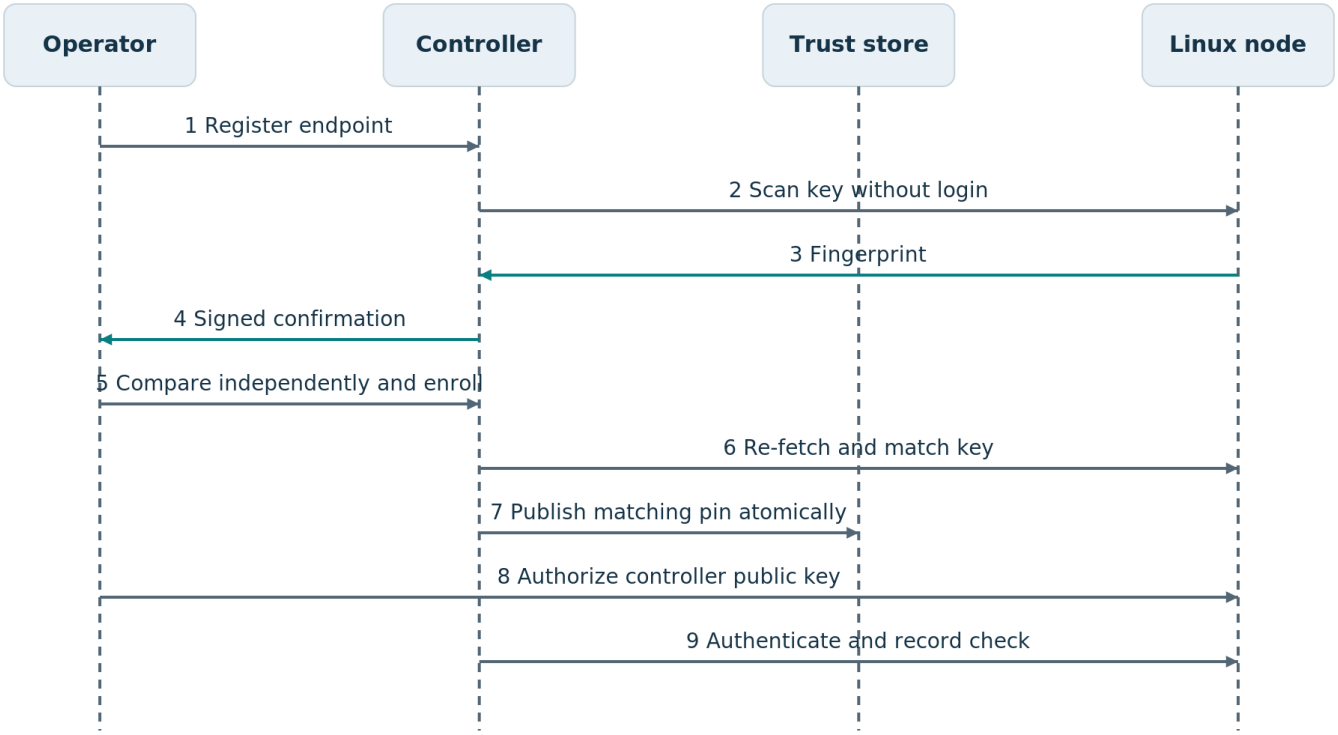


Figure 4. Onboarding sequence with separate server trust and client authorization.

POST /api/nodes creates an inventory record with HTTP 201; a duplicate endpoint returns 409. A host-key scan performs SSH key exchange without logging in. The operator compares the fingerprint through an independent trusted channel before enrollment. The controller public key is then authorized separately on the target.

READY describes validated setup for the current configuration revision. It is not a continuous reachability claim. An onboarding read can return HTTP 200 with CONFIGURATION_ERROR because the request succeeded while setup remains blocked.

Implementation anchors: inventory/router.py; connectivity/router.py, service.py, trust.py and readiness.py.

3.6 Trust consistency and configuration revisions

What the confirmation binds

The enrollment confirmation contains node ID, address, SSH port, configuration revision, fingerprint and issue time. HMAC-SHA256 signs that payload using a service-instance-local secret. Its validity window is zero to 300 seconds. Restart changes the secret and invalidates outstanding confirmations; confirmations are not consumed after use.

Atomic trust-store updates

Enrollment takes a cross-process sidecar lock and re-fetches the server key. A changed fingerprint, ambiguous store or conflicting existing pin rejects the enrollment. Even identical duplicate entries are treated as ambiguous. An identical unique pin succeeds without replacement. A new pin preserves existing content and is published through a temporary-file replacement with fsync and POSIX metadata handling.

Condition	Controller behavior	Reason
Key changes after scan	Reject and require a fresh scan	The approved identity no longer matches
Existing pin differs	Reject replacement in this workflow	Enrollment is not implicit key rotation
Existing unique pin matches	Succeed without replacing the file	The required trust relationship already exists
Configuration changes during a check	Persist observation without validating the new revision	Old credentials or trust state cannot validate new setup

Revision-aware setup

A check captures the configuration revision before transport execution and compares it again afterward. The revision binds endpoint and credential/trust identity. A result obtained under an old configuration remains useful history, but does not certify the replacement configuration.

Documented implementation defect: validate_confirmation uses assert for security checks. Optimized Python execution removes assertions. Run this implementation without -O or PYTHONOPTIMIZE; the code-level correction is explicit rejection checks with optimized-mode regression coverage. This report does not claim that correction has been applied.

Implementation anchors: ConnectivityService.confirmation and validate_confirmation; trust.py enrollment; readiness.py configuration_revision. Python optimization behavior: R4.

3.7 Availability scheduling and failure isolation

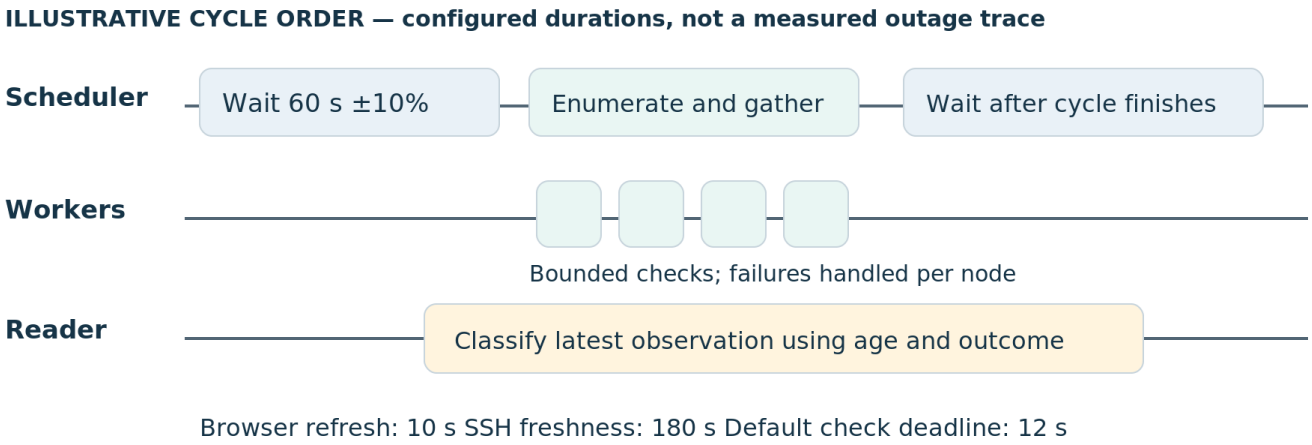


Figure 5. Scheduler timeline separating collection from read time freshness.

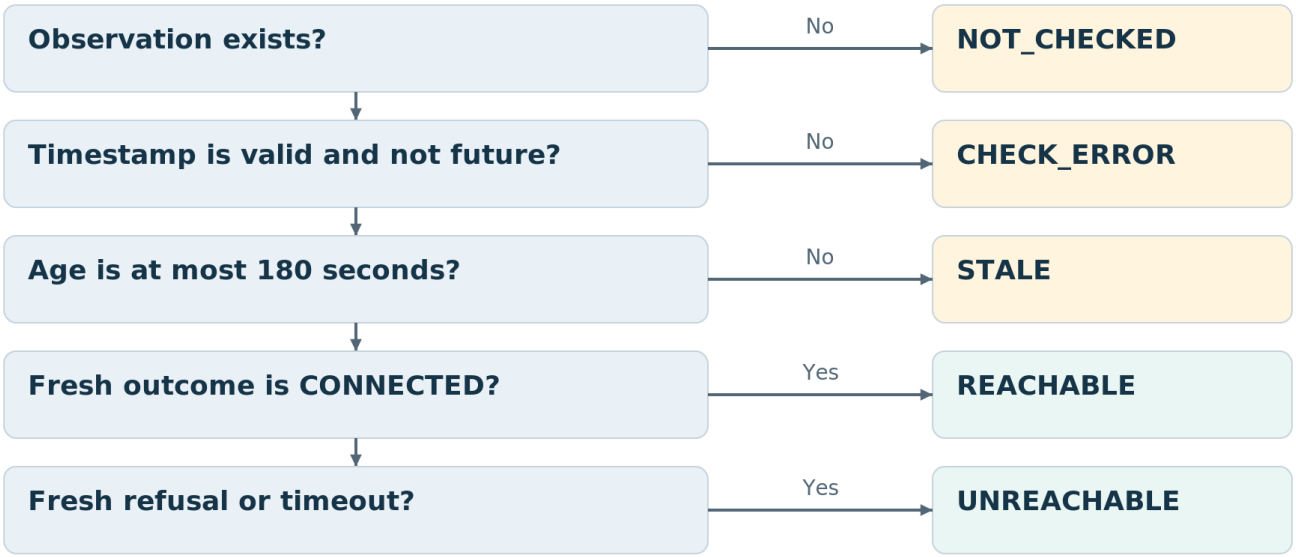
The scheduler runs as an in-process asyncio task. It waits for a jittered interval, reads inventory, creates bounded per-node checks and gathers results independently. The default interval is 60 seconds with ±10% jitter; the first sweep also waits unless an initial delay is configured. The next sleep begins after the current cycle finishes.

Boundary	Configured behavior	Consequence
Connectivity executor	Default capacity 4; total worker deadline 12 s	Slow targets consume bounded local capacity
Per-node work	In-flight tracking and admission exclusion	A busy check does not become a failed host observation
Global pressure	Admission rejection or skipped scheduling path	Old observations remain and can age naturally
Recovery	A later scheduled check records the returning endpoint	Connectivity recovery does not execute a repair command

The nominal interval is not a maximum detection time. Check duration, contention and scheduling add delay. A rough batching estimate is ceil(N / C) × T for N nodes, C workers and check duration T, before interval and overhead. It is a planning model, not a measured fleet throughput result.

Implementation anchors: connectivity/scheduler.py calculate_delay, run_cycle and _loop; connectivity/executor.py; config.py.

3.8 Availability and freshness interpretation



Continue downward on the other branch. Final other outcome: CHECK_ERROR.

Figure 6. Ordered availability decision flow.

The classifier evaluates missing data, timestamp validity and age before interpreting a connection outcome. CONNECTED maps to REACHABLE. CONNECTION_REFUSED and CONNECTION_TIMEOUT map to UNREACHABLE. Other fresh failures map to CHECK_ERROR, preserving authentication, trust, DNS and configuration distinctions.

A precise boundary example

If a successful observation was stored at 12:00:00 UTC and no newer check completes, it remains fresh at 12:03:00. At 12:03:01 it becomes STALE because age exceeds the 180-second TTL. That transition identifies expired evidence; it does not timestamp a host outage.

Keep three dimensions visible: setup readiness answers whether the current configuration has been validated; SSH status answers what the latest connection observation supports; resource health answers what usable telemetry exists.

Implementation anchors: connectivity/freshness.py derive_status; readiness.py derive_setup; metrics/health.py.

3.9 Controlled diagnostic catalog

The remote-operation interface deliberately exposes a small diagnostic catalog. The controller validates typed arguments and compiles a fixed command template. The browser never supplies an unrestricted shell program.

Action	Purpose	Input control
system.uptime	Inspect host uptime	Catalog-defined command
system.memory	Inspect memory information	Catalog-defined command
system.disk	Inspect filesystem usage	Disk path validation rejects traversal and whitespace
service.status	Inspect a named service	Service name follows a restricted pattern

Admission before execution

The executor checks shutdown state, same-node exclusion and global capacity under its lock. The default global operation capacity is four. A conflicting operation on the same node returns HTTP 409; a full global capacity returns 429. A rejected request has no operation row because it was not admitted for execution.

Output and result contract

A result includes outcome, failure category, exit code, bounded stdout and stderr, output_truncated, timestamps and operation ID. The truncation flag makes the output limit visible. A nonzero worker-process exit or malformed JSON is an infrastructure failure, distinct from the remote command's exit code. Worker exit code 2 maps to configuration failure.

Response	Meaning	Operator action
HTTP 200 with result record	The request produced a result; outcome can be FAILURE or TIMEOUT	Read outcome, category and output
409 or 429 before admission	No operation was admitted	Inspect contention or capacity
Admitted infrastructure error	Execution history may be uncertain and carry an operation ID	Look up the record before considering another run

Implementation anchors: operations/catalog.py validate_action; executor.py; transport.py; service.py; router.py.

3.10 Dispatch and the durable audit barrier

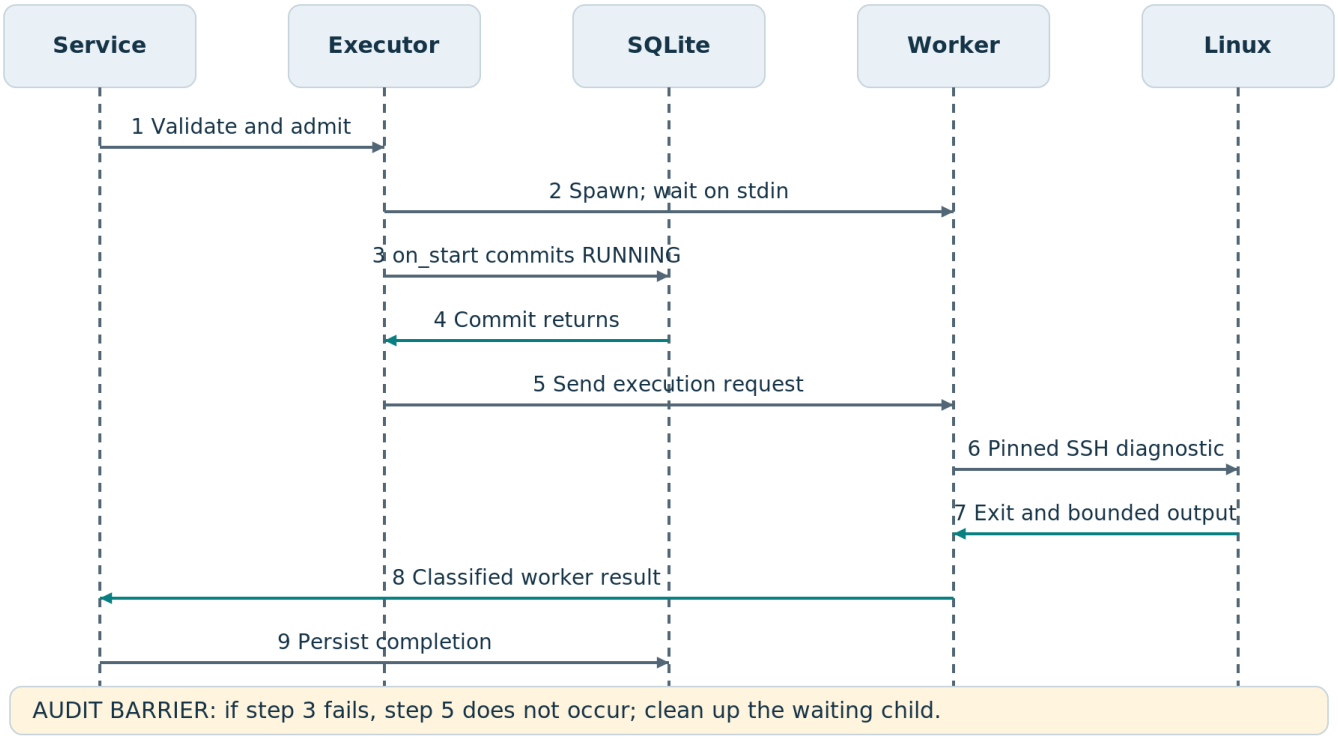


Figure 7. Operation sequence showing the durable audit barrier before dispatch.

The executor first spawns a local child with piped standard input and output, discarded stderr and an allowlisted environment. The child waits for its request. The on_start callback then commits the RUNNING operation record. Only after the callback returns does communicate send the execution request to the child.

Failure before dispatch

If worker creation fails, the callback does not run. If the initial audit write fails, the parent does not supply the remote command and attempts to kill and reap the waiting child. The distinction between spawning a worker and dispatching its input is central to the audit guarantee.

Deadline enforcement

The remaining deadline subtracts elapsed monotonic time since executor startup. A timeout produces TIMEOUT and triggers local termination and reaping. Local process cleanup bounds controller work; it does not establish that a remote process has stopped.

Implementation anchors: ProcessOperationExecutor.run and _terminate_and_reap; OperationService.execute; operation repository create.

3.11 Completion persistence and restart recovery

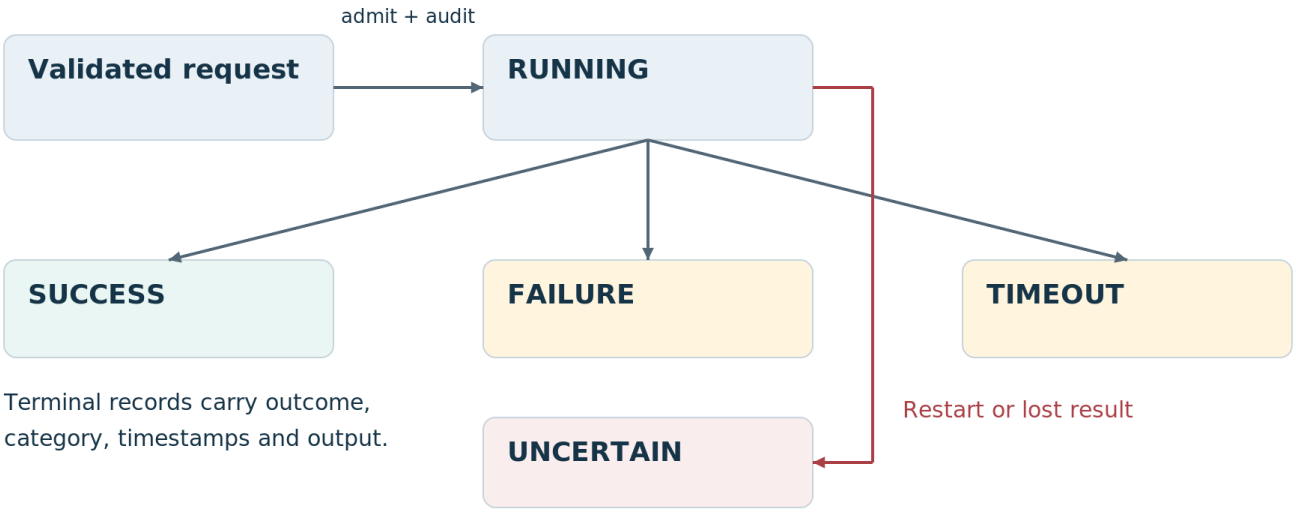


Figure 8. Operation outcome model and interrupted execution reconciliation.

After a worker returns, OperationService._complete attempts update_completion twice. If both writes fail, the service retains a process-local pending entry marked UNCERTAIN / UNAVAILABLE, preserves returned output fields and raises StorageError with the operation ID. The database can still contain RUNNING even though the worker returned a result.

Retry the record update

recover_completions retries the local completion write. It does not reopen SSH or send the action again. Once the write succeeds, the pending entry is removed and the conservative result is retained. Pending entries live in memory and are lost if the controller crashes.

Reconcile interrupted work

On startup, remaining RUNNING rows become UNCERTAIN / CONTROL_PLANE_RESTARTED with a completion timestamp. The controller does not reconstruct an unknown result as success and does not automatically replay the diagnostic. This prevents a lost response from being mistaken for an unexecuted command.

UNCERTAIN is an explicit application outcome. The implementation provides durable execution records, not exactly-once remote execution or a cryptographically immutable log.

Implementation anchors: operations/service.py _complete and recover_completions; repository.py reconcile_running; main.py.

3.12 Metrics collection and visualization

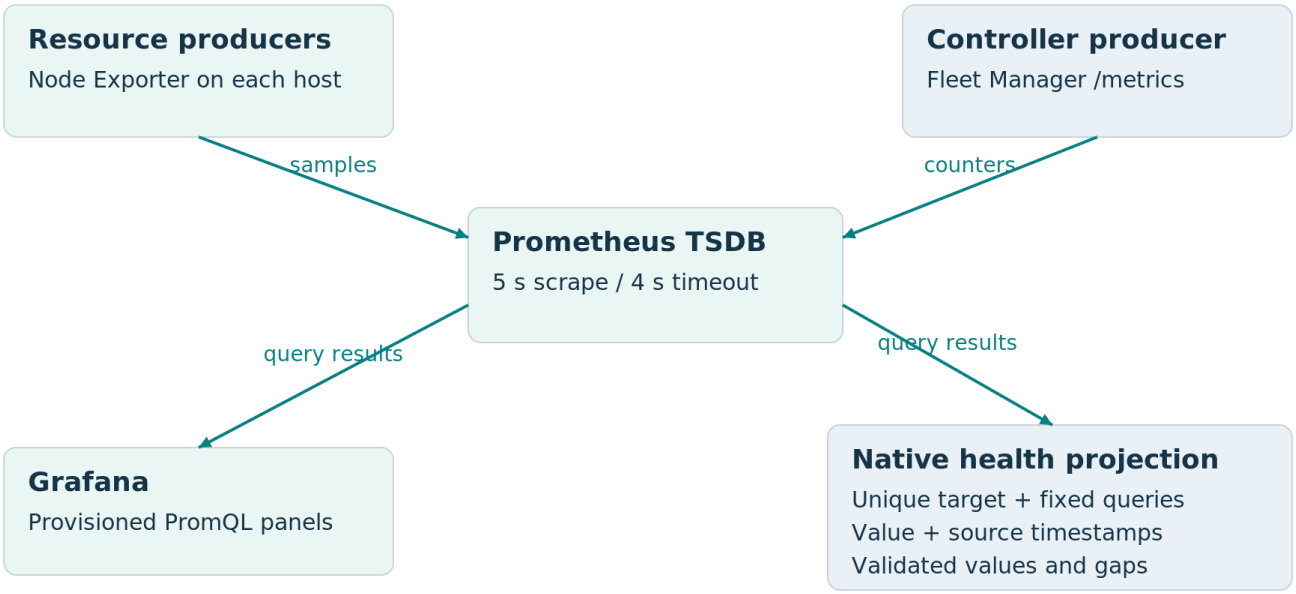


Figure 9. Telemetry data flow and separate visualization clients.

Node Exporter exposes Linux resource metrics directly to Prometheus. Fleet Manager /metrics exposes controller instrumentation and derived fleet counts; scraping that endpoint does not launch a new SSH probe. The monitoring configuration uses a five-second scrape/evaluation interval and a four-second scrape timeout, with file-based target discovery.

Two visualization clients

Grafana uses its provisioned datasource and dashboard queries. The native health endpoint resolves each inventory node to a unique owned exporter target, then executes a fixed expression catalog. It does not accept arbitrary PromQL. Missing or ambiguous target ownership returns unavailable rather than selecting an arbitrary series.

The catalog covers CPU utilization, available and total memory, root-filesystem capacity, uptime, load and network rates. CPU and network rates use five-minute windows. Network queries exclude loopback and selected virtual interfaces.

Implementation anchors: monitoring/prometheus/prometheus.yml; monitoring/grafana/; metrics/health.py METRICS; metrics/telemetry.py. Prometheus query semantics: R2.

3.13 Metric validity and source timestamps

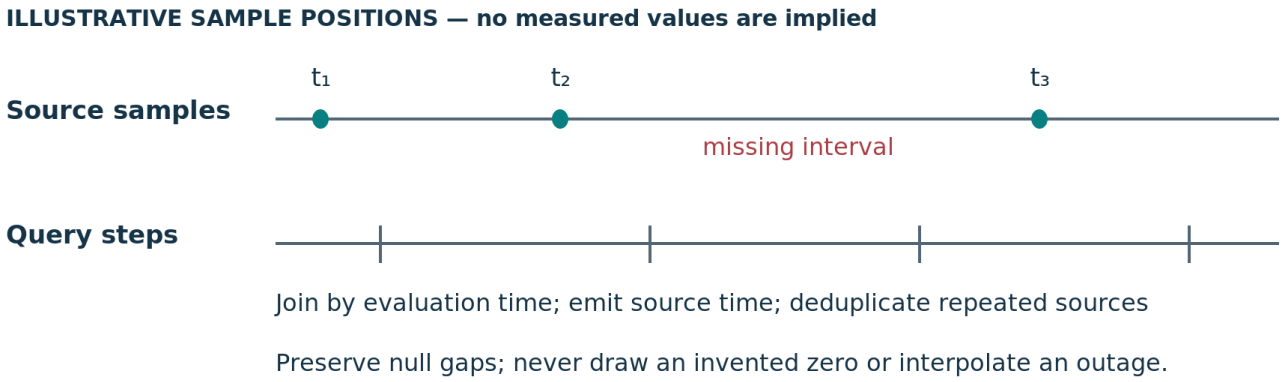


Figure 10. Source time and query time with explicit missing data.

A query evaluation time describes when Prometheus evaluated an expression. It does not necessarily identify when the underlying measurement was collected. The health adapter requests a value and a companion source-timestamp expression, joins them by evaluation time and emits the source timestamp.

Freshness rules

For multi-selector expressions, _source chooses the oldest current contributing source sample. This is a conservative indication for the current selectors, not the age of every sample in a five-minute rate window. The resource freshness threshold is 90 seconds, separate from the 180-second SSH observation TTL.

Input condition	Health projection
Fresh target with up = 0	Down; do not fabricate resource values
Source older than freshness threshold	Stale
Ambiguous target or invalid data	Unavailable
Invalid history measurement with a valid source position	Null gap at that position
Repeated source timestamps across query steps	Deduplicate observations

History spans one hour with 150-second query steps. Nonfinite or negative values, percentages above 100, future timestamps and inconsistent capacity pairs are rejected. The chart leaves gaps disconnected. A displayed history count is therefore not an exporter scrape count.

Implementation anchors: metrics/health.py _source, _sample, _observation and _node; browser health/chart modules. Reference R2.

3.14 Relational model and storage ownership

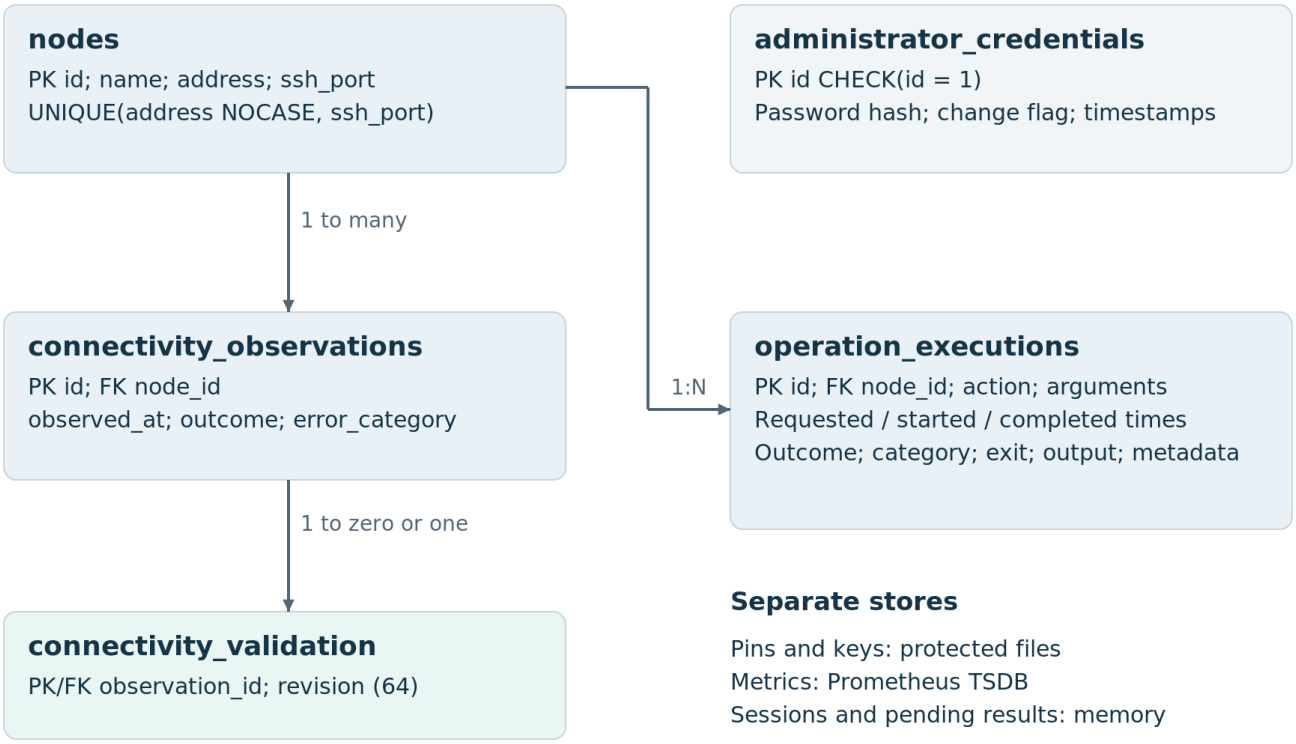


Figure 11. Relational data model with exact ownership and cardinalities.

The nodes table enforces endpoint uniqueness using case-insensitive address and SSH port, along with nonempty identity fields and valid integer ports. Connectivity observations reference nodes; an optional validation row associates an observation with a 64-character configuration revision.

Operation rows contain sanitized arguments, request/start/completion timestamps, outcome, failure category, exit code, bounded output, truncation and audit metadata. Completion updates the existing row. administrator_credentials is a singleton with id = 1 and has no foreign key linking it to operation attribution.

Repositories use parameterized SQL, foreign-key enforcement and short transactions. Initialization validates schema version and applies explicit migration steps. WAL supports the compact controller’s concurrency pattern; SQLite writes remain a serialized resource.

Implementation anchors: inventory, connectivity, operations and security repository schemas; scripts/backup_restore.py.

3.15 Authentication and browser sessions

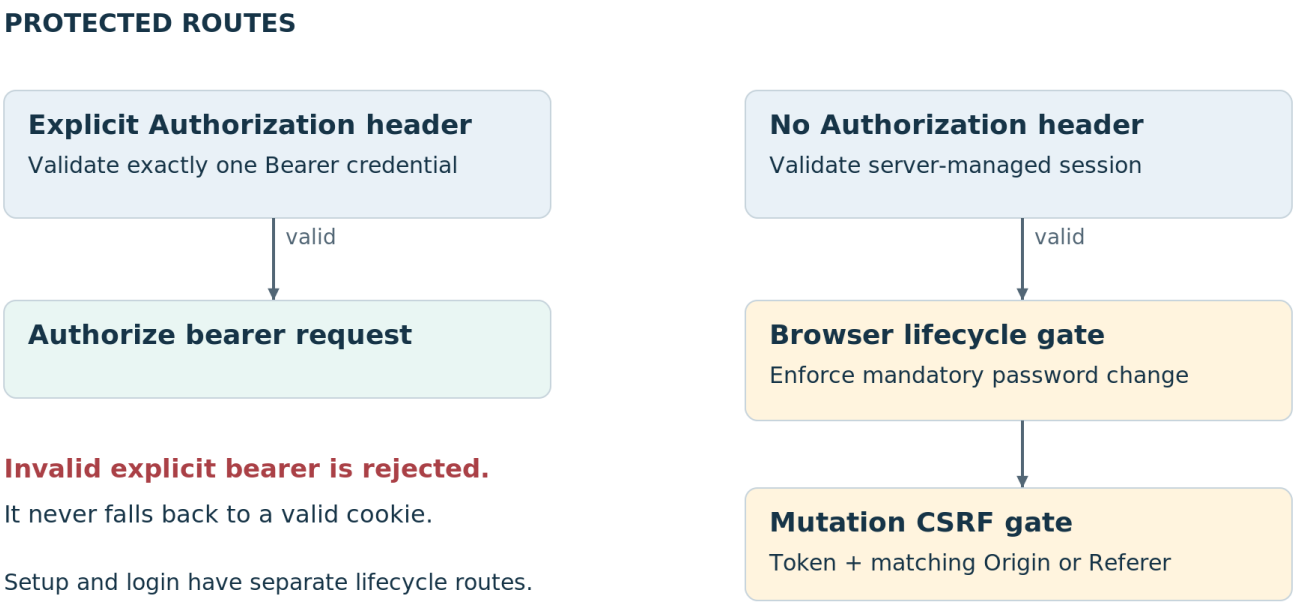


Figure 12. Authentication branches and browser mutation gates.

An explicit Authorization header selects bearer authentication. The gate accepts exactly one well-formed Bearer credential and compares digests in constant time. An invalid bearer cannot fall back to a valid browser cookie. A valid bearer returns before cookie-specific password-change and CSRF checks.

Without that header, the request requires a valid server-managed session. Idle expiry defaults to 30 minutes and absolute expiry to 12 hours, tracked with monotonic time. The store retains a token verifier and server-side CSRF material. Session lookup refreshes activity before later route authorization checks.

Cookie-authenticated protected mutations require a synchronizer token and matching Origin, or Referer when Origin is absent. Missing origin information rejects the mutation. The cookie is HttpOnly and SameSite=Lax; Secure is set for HTTPS requests. Password hashes use Argon2. Sessions and login throttling remain process-local.

The frontend sends same-origin credentials and CSRF tokens on mutations. Authentication failure prompts for login without automatically replaying a remote diagnostic.

Implementation anchors: security/auth.py require_operator_token; session.py SessionStore and validate_csrf; passwords.py; ui/static/js/api.js.

3.16 Security controls and engineering limits

The security model combines authenticated access, explicit host identity, a restricted command catalog and bounded execution. These controls protect different boundaries; none substitutes for the others.

Threat or failure	Implemented control	Remaining boundary
Unauthorized request	Bearer or browser-session gate	One administrative identity; no per-person RBAC
Cross-site mutation	CSRF token plus Origin/Referer validation	Setup/login have separate lifecycle routes
Host impersonation	Independent fingerprint comparison and pinned identity	Confirmation assert defect described in 3.6
Shell injection	Fixed templates, typed arguments and quoting	Target account privileges and executable behavior
Resource exhaustion	Capacity limits, deadlines and bounded output	No distributed rate-limit or capacity SLA
Lost remote completion	Operation lookup, UNCERTAIN and no replay	Exactly-once effects and remote termination are not guaranteed
Audit modification	Persistent records and filesystem permissions	Privileged database access can change history
Controller failure	Service restart and RUNNING reconciliation	Single control-plane failure domain; sessions are lost

Operational constraints

Use non-root target credentials and protect the private key, host pins, database and configuration. Keep the reference deployment's loopback/tunnel access model explicit. HTTPS and proxy scheme handling determine transport protection; a cookie flag alone does not encrypt HTTP.

Prioritized engineering work

Replace confirmation assertions with explicit checks. Introduce named identities when accountability requires them. Add durable work ownership before multiple controller workers. Off-host audit retention and backup authentication are separate extensions, not properties of the current local database.

Implementation anchors: security modules; connectivity/trust.py; operation catalog and executor; deploy/fleet-manager.service. Python assert behavior: R4.

3.17 Backup and recovery procedures

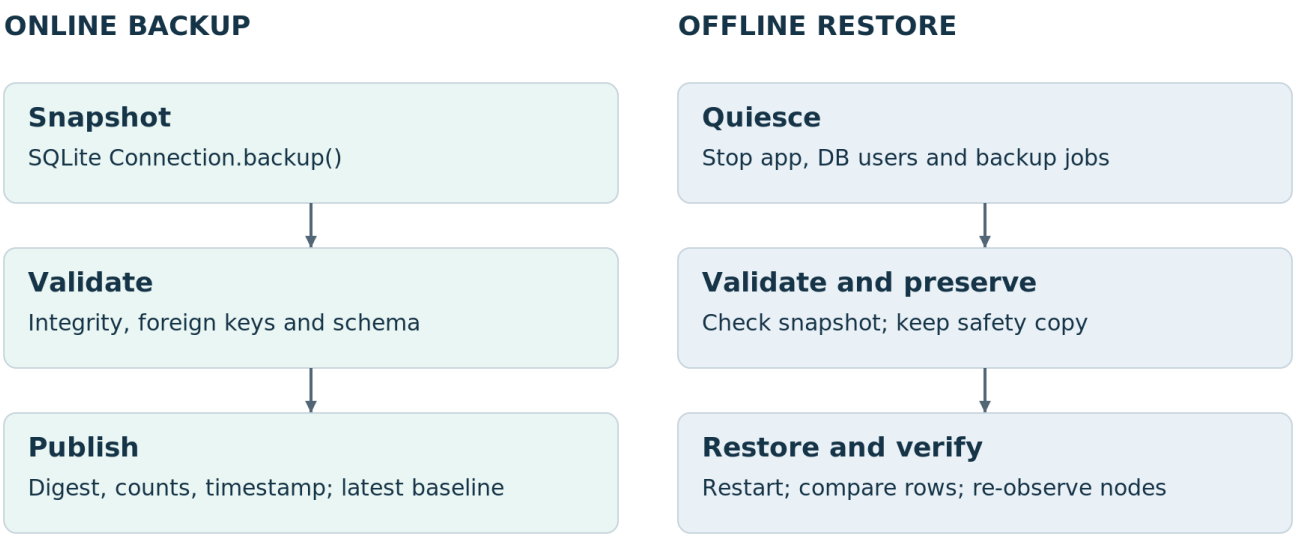


Figure 13. Paired backup and restore procedures with independent asset boundaries.

The backup utility uses sqlite3.Connection.backup(), validates integrity, foreign keys and schema, and creates a manifest containing digest, row counts and timestamp. It publishes the snapshot through replacement. Scheduled retention maintains one latest baseline. It does not provide multi-generation retention.

Restore is explicitly offline. Stop database users and backup jobs, confirm no handles remain, validate the snapshot and preserve a pre-restore safety copy. A safety-copy collision aborts the restore. After restart, compare inventory and operation history, inspect reconciliation outcomes and re-observe nodes.

Recovery concern	Mechanism
Network connectivity returns	Next scheduled check produces a fresh observation
Controller process restarts	Service restart; initialization and operation reconciliation
Completion write fails	Local persistence retry without remote dispatch
Control database is lost or corrupt	Validated snapshot and offline restore
Keys, pins or telemetry storage are lost	Separate asset-specific recovery; outside the database snapshot

Implementation anchors: scripts/backup_restore.py; backup units; docs/DEPLOYMENT-RUNBOOK.md. SQLite Online Backup API: R3. No numerical RPO or RTO is assigned.

3.18 Operator workflow and interface behavior

The dashboard organizes work around fleet overview, selected-node context, diagnostic execution and result inspection. The operator can move from a status summary to setup guidance or operation history without using a remote terminal.

Stage	Visible information	Purpose
Orient	Fleet summaries, setup states, observations and health	Identify nodes requiring attention
Investigate	Endpoint, timestamp, error reason and resource history	Locate the failing boundary
Act	Registration, trust enrollment and fixed diagnostic choices	Perform a deliberate, bounded action
Verify	Operation ID, outcome, output and history	Check what happened before another action

Progressive disclosure

Summary information remains visible while raw output and detailed history are available on demand. The operation workbench shows a command preview and selected action. Setup guidance remains beside trust and authentication state; freshness remains beside the observation it describes.

Refresh and context

The authenticated browser refreshes fleet information every ten seconds. This refresh reads state; it does not imply that SSH or exporter data was measured at that instant. Node selection and measurement timestamps give the operator continuity while new observations arrive.

Interaction controls

Named dialogs, focus ownership, background inert state, focus restoration, explicit labels and live status messages support the workflow. Text labels distinguish outcomes without relying only on color. These are implemented interaction mechanisms; they are not a claim of a formal accessibility certification.

Implementation anchors: dashboard.html; app.js; fleet.js; node-onboarding.js; operations.js; history modules; dialog-ownership.js.

4 OUTPUT AND SCREENSHOTS

4.1 Inventory and node setup

Retained browser-QA capture using controlled SSH and Prometheus fixtures.

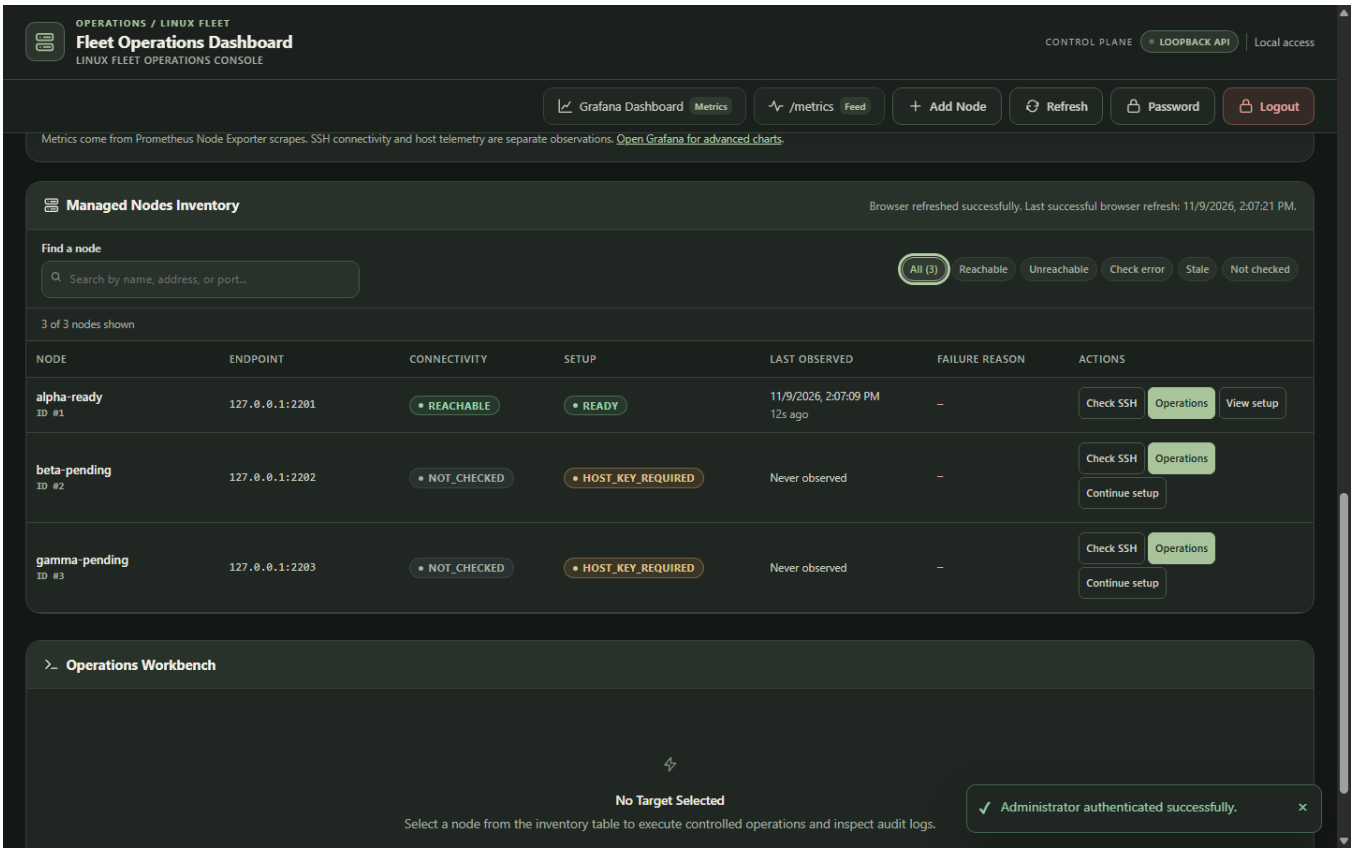


Figure 15. Inventory view with node endpoints, setup status, connection observations and available actions.

What the interface communicates

The inventory view places endpoint identity, connectivity, setup status and observation time in the same row. A node can require host-key enrollment while another node is ready for diagnostics. The controls lead to connectivity checks, operations and setup guidance.

The displayed alpha, beta and gamma nodes belong to the controlled browser-QA environment. The screen documents how distinct setup states are presented; the loopback endpoints and fixture labels are preserved from the original capture.

Original screenshot extracted without alteration from supplied report (6), pages 42–43.

4.2 Diagnostic workbench

Retained browser-QA capture using controlled SSH and Prometheus fixtures.

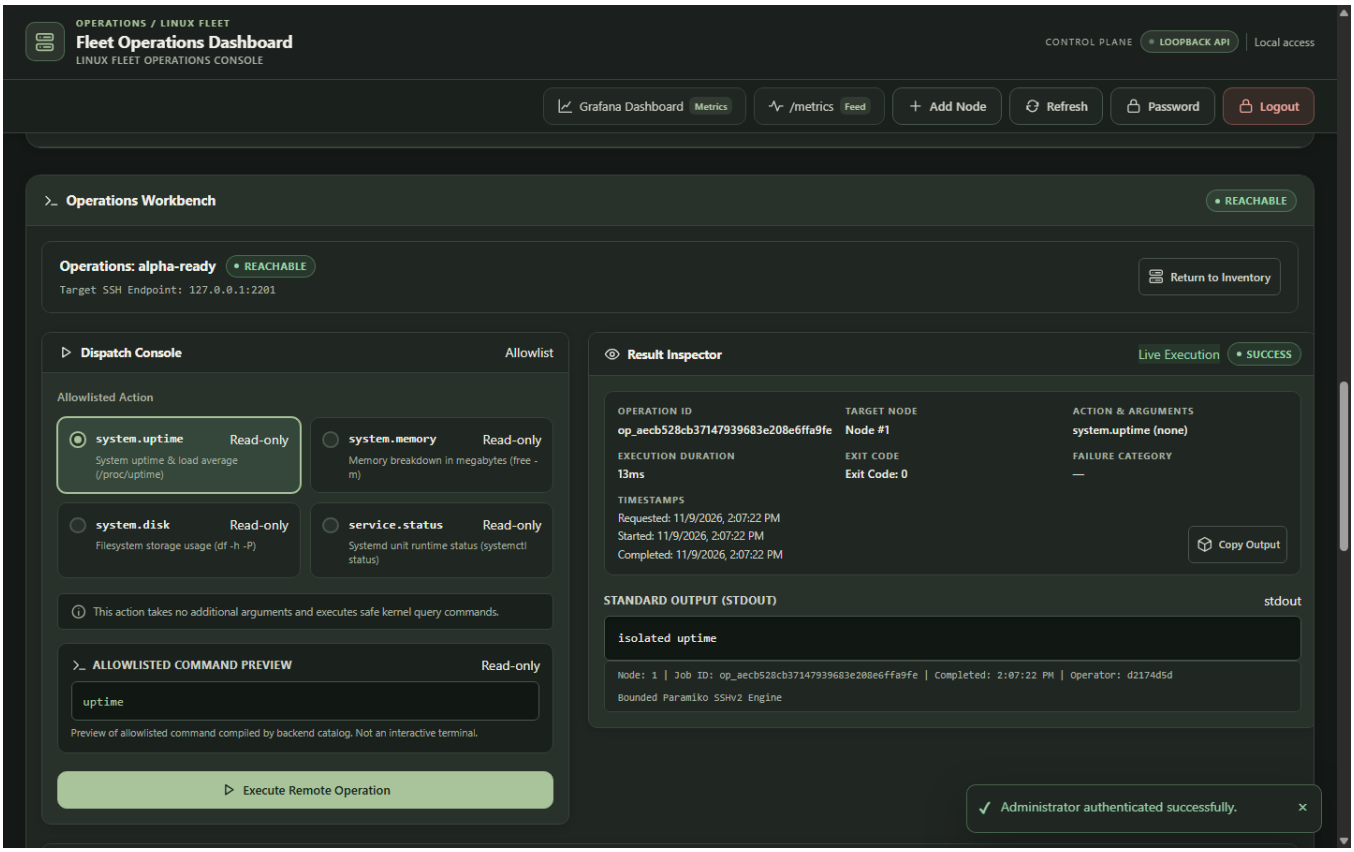


Figure 16. Fixed diagnostic actions, command preview and result inspection in browser QA.

What the interface communicates

The workbench keeps action selection and result inspection together. The operator selects one of the four diagnostic actions, reviews the compiled command preview and executes it for the selected node. The result pane presents operation identity, execution outcome and output.

Displayed command output is a QA fixture. In the application workflow, the operation ID connects the response to persisted history. An HTTP response alone is not enough to classify a diagnostic; the operator reads outcome, category and output.

Original screenshot extracted without alteration from supplied report (6), pages 42–43.

4.3 Native resource trends

Retained browser-QA capture using controlled SSH and Prometheus fixtures.

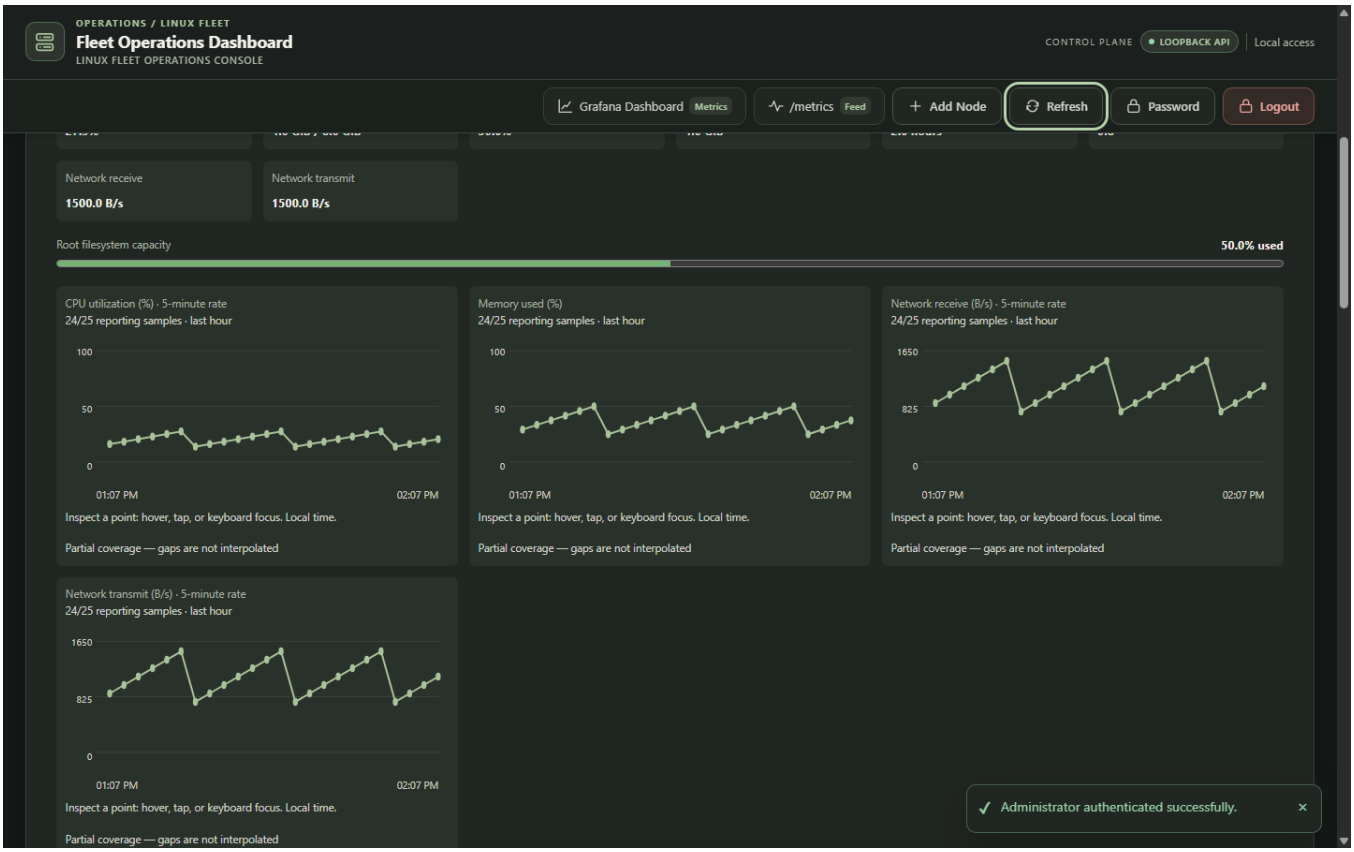


Figure 17. Native Fleet Health trends for CPU, memory and network in browser QA.

What the interface communicates

The native health interface presents resource summaries and time-series charts for the selected node. CPU and network values derive from rate windows; memory and filesystem values retain their capacity context. Source timestamps identify the observations represented.

This is the controller's health interface, not Grafana. Both use Prometheus through distinct query and presentation paths. The supplied capture uses controlled Prometheus fixtures to exercise chart rendering and interaction.

Original screenshot extracted without alteration from supplied report (6), pages 42–43.

4.4 Missing measurements

Retained browser-QA capture using controlled SSH and Prometheus fixtures.

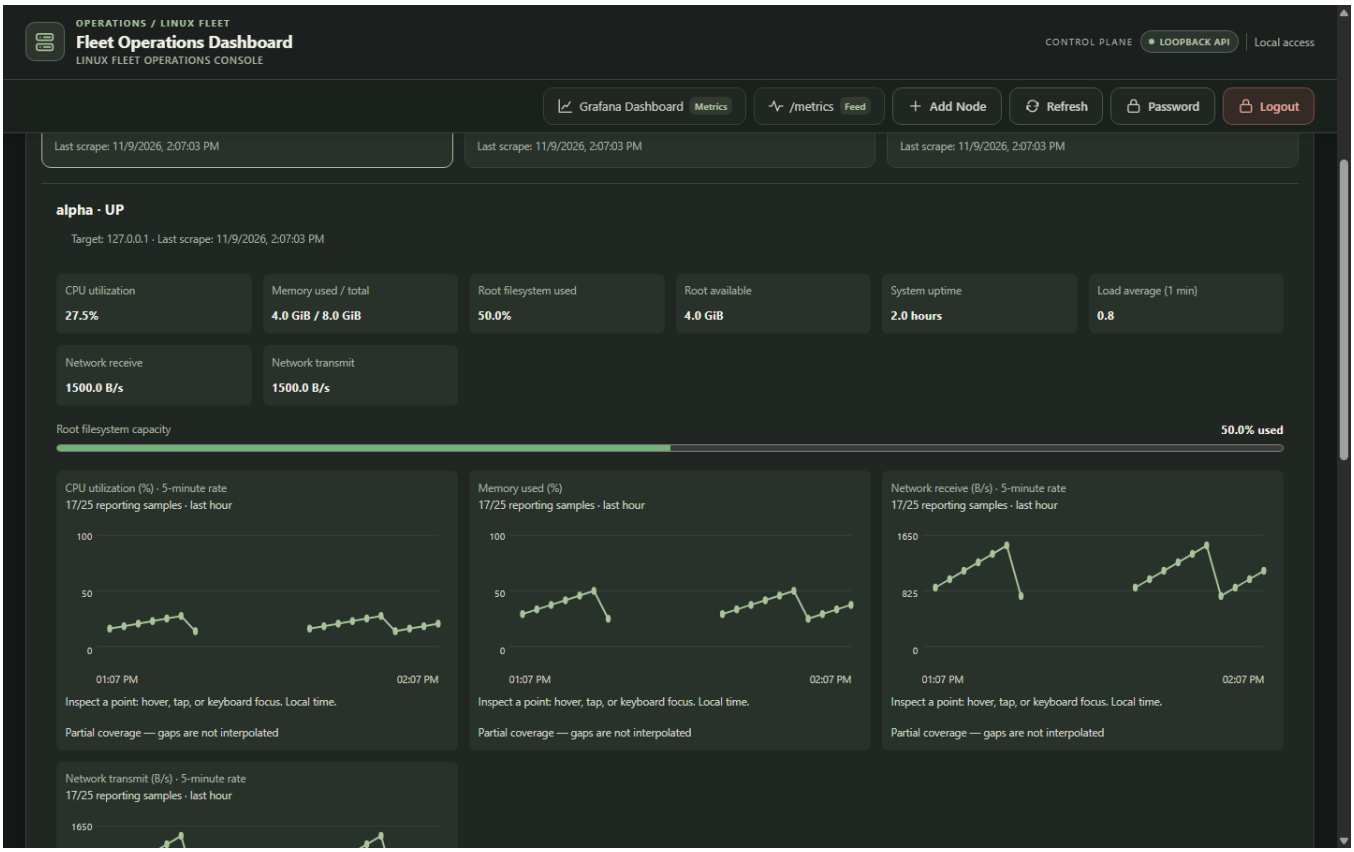


Figure 18. Missing-data presentation preserves disconnected series.

What the interface communicates

Disconnected chart segments mark missing measurements. The application preserves null gaps rather than inserting zeros or drawing a continuous line across an interval with no valid data. This keeps the visual history consistent with the health adapter's validity rules.

The capture exercises missing-data behavior with controlled QA data. It illustrates the interface contract: a measurement gap is visible as a gap, and the selected node's identity and available resource context remain on screen.

Original screenshot extracted without alteration from supplied report (6), pages 42–43.

5.1 Test strategy and recorded results

Verification covers backend contracts, frontend modules, browser workflows, authentication, operation behavior and bounded admission. The table preserves the dates and units of the recorded results so repeated test executions are not counted as additional unique tests.

Layer and date	Recorded result	Coverage or context
Python suite · 9 Nov	535 passed; 7 warnings 150.55 s	Backend, security and persistence contracts
Frontend · 9 Nov	74 passing executions 37 unique tests	Repeated through compatibility entry points
Browser QA · 9 Nov	126 assertions; cleanup true	Normal workflow passed on retry with fixtures
Authentication · 11 Nov	25 passed; 7 warnings 4.70 s	Focused authentication regressions
Operation units · 11 Nov	33 passed; 1 warning 1.65 s	Focused operation behavior
Retained acceptance run	2 local passes; 6 skipped; 6 simulated	Explicit mock invocation; exit 2

Failure contracts exercised by the design

Scenario	Expected system response
Unknown or changed host key	Block implicit trust and present enrollment guidance
Authentication or DNS failure	Persist a classified observation
Capacity saturation	Reject excess work and release admitted capacity after completion
Timeout or transport loss	Retain classified result or UNCERTAIN; do not blindly replay
Controller restart	Reconcile RUNNING to UNCERTAIN
Missing metric data	Return stale/unavailable or a null gap; never fabricate zero

The first normal browser invocation failed with a Windows PermissionError before the driver launched; the retry passed. The project author confirms implementation and testing completion. The numerical records above retain their original execution context rather than being rewritten as new runs.

Recorded sources named in the supplied reports: final_python.txt; browser-QA results; authentication and operation runs; acceptance-final.json and .txt.

5.2 Monitoring and admission measurements

Prometheus scrape snapshot on 8 November 2026

Target	Instance	Duration ms	Health
Fleet Manager	127.0.0.1:8000	8.998	up
Node A exporter	10.160.0.5:9100	132.629	up
Node B exporter	10.160.0.3:9100	126.651	up
Node C exporter	10.160.0.4:9100	119.757	up
Prometheus	localhost:9090	6.118	up

The retained response records all five targets up with empty lastError and a five-second scrape interval. Last-scrape times fall between 09:49:10.058 and 09:49:12.432 UTC. This is a dated scrape snapshot, not a continuous-availability percentage.

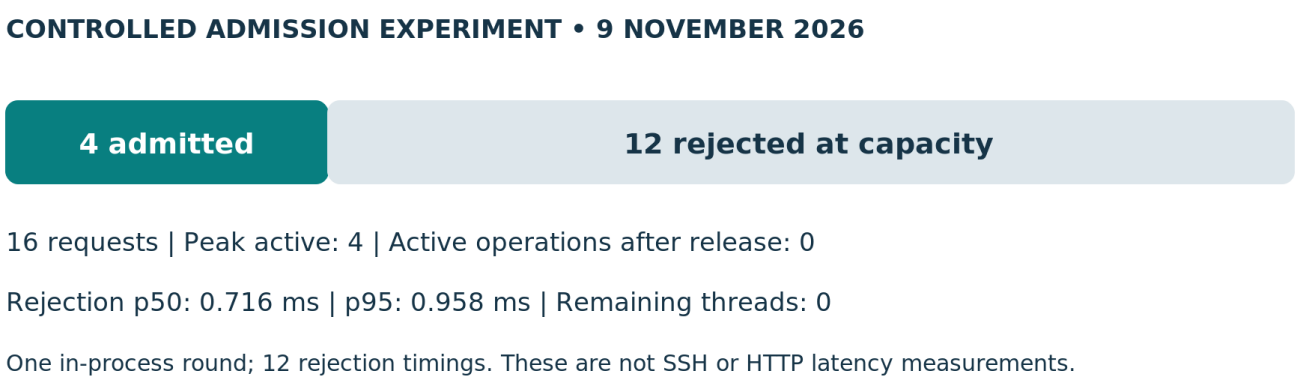


Figure 14. Measured admission and cleanup results.

The admission experiment filled four operation slots, held them open, attempted twelve additional requests and then released the admitted work. Timing used time.perf_counter_ns from service entry to rejection. The single round had no warmup and used controlled in-process transport.

The result demonstrates bounded admission and cleanup for that workload. The p50 and p95 describe twelve rejection timings; they do not measure remote execution latency or establish a maximum fleet size.

Recorded sources: controller-validation.txt, run 20261108T1515; admission-baef7aca7ebf4b3b80383bf99aa722b1/results.json. Values transcribed from both supplied reports.

ENGINEERING RATIONALE

6 Design decisions and tradeoffs

Decision	Why it fits	Cost or consequence
Modular FastAPI controller	Keeps HTTP contracts, use cases and persistence in one deployable application	Module boundaries require discipline; one controller failure domain
Paramiko and local child workers	Combines pinned SSH with bounded local process lifetime	Process overhead; cleanup cannot prove remote termination
SQLite WAL	Compact transactional control store and straightforward backup	Serialized writes; not a shared distributed ownership layer
Prometheus and Grafana	Dedicated metric collection, storage and visualization	Target identity, provisioning and access paths need separate configuration
Separate setup and availability	Prevents old setup history from being treated as current reachability	More states must be explained clearly
Four fixed diagnostic actions	Reduces unsafe input and makes result contracts testable	New operations require catalog and validation changes
No automatic remote replay	Avoids duplicate work after a lost result	Operator investigation is required for uncertain completion
Browser ES modules	Small client dependency surface and explicit feature modules	Focus and state coordination need regression coverage

Scaling path

Measure representative HTTP latency, SSH duration, sweep completion time, rejection rate and database contention first. Tune within the single-worker contract. If workload requires multiple controllers, introduce durable ownership or leases, shared session decisions and a storage design that supports those semantics before adding workers.

Engineering result

The project's depth lies in its boundary handling: revision-aware trust validation, audit-before-dispatch, isolated execution, explicit completion states, source-time-aware metrics and separate recovery procedures. These mechanisms make failure behavior inspectable and explainable at the API, storage and operator-interface levels.

Rationale synthesized from the implemented mechanisms and supplied reports. Existing project ADRs remain in docs/decisions/0001–0007.

OPERATIONAL REFERENCE

Appendix A Deployment and reproduction

Deployment sequence

Step	Action and completion check
1 Prepare hosts	Configure independent Linux nodes, sshd, non-root login accounts and Node Exporter. Record host identities and network endpoints.
2 Prepare controller	Install locked Python 3.12 dependencies. Create protected configuration and state paths. Configure the single-worker service.
3 Configure monitoring	Set Prometheus discovery targets and Grafana provisioning. Check service listeners and the operator access path.
4 Enroll nodes	Register inventory, compare fingerprints independently, enroll pins and authorize the controller public key on targets.
5 Observe and diagnose	Check authenticated observations, exporter targets, time-range samples and rendered dashboards. Execute catalog diagnostics and inspect operation history.
6 Exercise recovery	Use the project runbook for controlled interruption and restore procedures. Preserve target, timestamp, result and recovery context.

Local verification commands

```
uv sync --locked
uv run --no-sync python -m pytest -q
node --experimental-vm-modules --test \
  tests/dashboard_modules.cjs \
  tests/dashboard_regression.cjs
uv run --no-sync python -m ruff check src tests scripts
uv run --no-sync python -m mypy src
```

Run these commands from the application repository using its locked environment. They reproduce the project checks; creating this document does not execute them against the application. The deployment runbook remains the authority for service configuration, access and restore steps.

Implementation anchors: docs/DEPLOYMENT-RUNBOOK.md; deploy/gce/; deployment and backup systemd units.

SOURCE INDEX

Appendix B Implementation traceability

The source map identifies the application locations associated with each mechanism. Paths below are reproduced from the supplied implementation reports. Module-family paths are relative to src/fleet_manager/ unless explicitly stated otherwise.

Report topic	Implementation anchor
Scope and requirements	AGENTS.md; docs/PROJECT-CHARTER.md
Lifecycle and deployment	main.py; deploy/fleet-manager.service
Inventory and endpoint uniqueness	inventory/router.py; inventory/repository.py
Enrollment and key publication	connectivity/router.py; service.py; trust.py
Setup revisions and display status	connectivity/readiness.py; freshness.py; models.py
Periodic checks and capacity	connectivity/scheduler.py; executor.py; config.py
Action validation and dispatch	operations/catalog.py; executor.py; transport.py
Audit completion and reconciliation	operations/service.py; operations/repository.py
Protected route authentication	security/auth.py; session.py; router.py; passwords.py
Resource queries and instrumentation	metrics/health.py; health_router.py; telemetry.py
Prometheus and Grafana	monitoring/prometheus/; monitoring/grafana/
Operator interface	ui/templates/dashboard.html; ui/static/js/ modules
Backup and restore	scripts/backup_restore.py; deployment runbook
Design records	docs/decisions/0001–0007

Document basis

This consolidated report combines the 24-page engineering edition (5) and the 46-page implementation-depth edition (6), both dated 11 November 2026. The deeper mechanisms from D1–D10 are integrated into the main narrative. Repeated overview material and documentation-build commentary have been removed. The author’s confirmation establishes completion status; original counts, screenshots and implementation limits are preserved accurately.

REFERENCE MATERIAL

Appendix C References and terminology

Research used for diagram and technical presentation

R1. Simon Brown. C4 model: Diagrams, Deployment diagram and Dynamic diagram. Used to separate system context, deployment boundaries and event order. https://c4model.com/diagrams · https://c4model.com/diagrams/deployment · https://c4model.com/diagrams/dynamic

R2. Prometheus. Querying basics. Used to distinguish instant and range evaluation and to explain query time separately from measurement context. https://prometheus.io/docs/prometheus/latest/querying/basics/

R3. SQLite. SQLite Backup API. Used to explain a database snapshot created through the online backup API. https://sqlite.org/backup.html

R4. Python documentation. Command line and environment, optimization options. Used to verify that -O removes assert statements. https://docs.python.org/3.11/using/cmdline.html

References consulted on 12 November 2026. External documentation informs the explanation; project-specific behavior and measurements come from the supplied reports and author confirmation.

Terminology

Term	Meaning in this report
Node	A registered Linux machine endpoint managed by the controller
Setup readiness	Validation history associated with the current configuration revision
Observation	A timestamped result of an SSH connectivity check
Source timestamp	Time attached to the underlying metric sample used by the health projection
Admission	Permission for work to occupy bounded executor capacity
Audit barrier	Durable RUNNING record committed before execution input reaches the worker
UNCERTAIN	An operation outcome whose completion cannot be established safely
QA fixture	Controlled test data used to exercise interface or application behavior